TheBeetamin — Complete Project Documentation

Generated from codebase review (read-only).
Version: As of repository snapshot.
Do not treat pricing figures as legal or contractual commitments — verify with each vendor.

---

1. Project Overview

What this app does (simple words)

TheBeetamin is a **Next.js web application** that:

- Presents a **marketing landing page** for a nutrition coaching product (₹3,999 multi-session plan, free assessment, etc.).
- Lets **visitors take a free health assessment** (questions + AI-generated results via Groq).
- Lets **patients sign in with Clerk**, complete **profile onboarding**, and **book appointments** with nutritionists using availability slots stored in Supabase.
- Lets **whitelisted nutritionists sign in with Supabase Auth** (email/password) and manage **appointments** and **availability** on a dedicated dashboard.
- Supports a **₹29 lead / purchase flow** (assessment purchase page + API lead capture + email notification).

Who the users are

Role	Auth system	Primary surfaces
Patient (end user)	**Clerk** (e.g. Google OAuth / Clerk UI)	Landing, assessment, booking, profile, session booking
Nutritionist (admin)	**Supabase Auth** (not Clerk)	Nutritionist sign-in tab → dashboard & availability

Important: The **main site Navbar** is built for **Clerk patients only**. Nutritionists use Supabase auth and typically land on `/sign-in` (nutritionist tab) → `/nutritionist-dashboard`, not the marketing Navbar variant.

Complete user journeys (high level)

Patient — discovery → booking

1. **Landing** (`/`) → sections: hero, experts, pricing, assessment CTA, FAQ, etc.
2. **Optional: Assessment** (`/assessment`) → **Results** (`/assessment/results`) stored in `localStorage`.
3. **₹29 path:** Results or purchase → **`/assessment/purchase`** (Clerk gate) → simulated lead save via **`/api/save-lead`** → confirmation UI.
4. **₹3,999 path: Booking entry** (`/booking`) → Clerk sign-in → hub redirects: new user → **`/booking/onboard`** (profile setup) → **`/booking/dashboard`** or **`/booking/new`** for booking; returning user with client row → **`/booking/dashboard`**.
5. **Book session** (`/booking/new`) → **`/booking/success`** after scheduling.
6. **Profile / sessions:** **`/booking/profile`**, **`/booking/dashboard`**.

Nutritionist

1. **`/sign-in`** → **"I'm a Nutritionist"** tab → email **whitelist check** (`lib/nutritionist-config.ts`) → password → **Supabase** `signInWithPassword`.
2. Client sets **`nut-email`** **cookie** for middleware visibility; middleware allows only nutritionist routes.

3. `/nutritionist-dashboard` — appointments; `/nutritionist-dashboard/availability` — weekly slots.

---

2. Complete Tech Stack

For **each** dependency in `package.json` (runtime):

Tool	Role in *this* project	Typical files / areas
Next.js 16	App Router, pages, API routes, SSR/CSR	`app/**`, `next.config.ts`
React 19	UI	`app/**`, `components/**`
TypeScript	Typing	All `.ts` / `.tsx`
Tailwind CSS 4	Styling	`app/globals.css`, classNames across UI
@clerk/nextjs	Patient authentication, `<SignIn/>`, `useUser`, `auth()` in server actions	`app/layout.tsx`, `app/sign-in/**`, `lib/booking-actions.ts`, `proxy.ts`
@supabase/supabase-js	Browser Supabase client; admin client with service role	`lib/supabase.ts`, `lib/supabase-admin.ts`, nutritionist pages
framer-motion	Animations	`components/sections/*`, many pages
lucide-react	Icons	Throughout UI
groq-sdk	AI assessment report generation	`app/api/assessment/route.ts`
resend	Transactional email (leads, reminders flow)	`app/api/save-lead/route.ts`, `lib/booking-actions.ts`
class-variance-authority, **clsx**, **tailwind-merge**	Component variant / class utilities	`components/ui/*`, `lib/utils.ts`
radix-ui (meta package)	UI primitives (where used)	Check imports in `components/`
shadcn (CLI tooling)	Project scaffolding	`components.json`

Infrastructure / services (not all in package.json)

Service	Role in *this* project	Where it appears
Supabase	Postgres DB + **Auth for nutritionists**; accessed via anon + **service role** server-side	`lib/supabase.ts`, `lib/supabase-admin.ts`, `lib/*-actions.ts`, `SUPABASE_SETUP.sql`
Clerk	**Patient** identity	`ClerkProvider`, `proxy.ts`, sign-in page
Groq	LLM API for assessment JSON	`app/api/assessment/route.ts`, `GROQ_API_KEY`

Resend	Outbound email	`RESEND_*` env vars, API routes / booking-actions
Vercel (typical)	Hosting; `vercel.json` defines a **cron** hitting reminders API	`vercel.json`
Cal.com (optional integration)	Env vars suggest scheduling integration may exist	`.env.local` patterns (user environment)

Free tier, paid tier, and rough monthly cost

Important: The following are **typical vendor public tiers** as of common knowledge; **confirm on each vendor's pricing page** before budgeting.

Service	Free tier (typical)	Paid triggers	Rough scale cost (indicative)
Vercel	Hobby project limits	Team features, bandwidth, enterprise	**100 users:** often $0–20/mo hobby; **1k–10k:** Pro ~$20+/mo + usage
Supabase	Free tier DB/auth limits	Storage, DB size, MAU for auth, projects	**100:** $0–25; **1k–10k:** often **Pro ~$25/mo** + overages
Clerk	MAU-limited dev/free tiers	MAU, organizations	Scales roughly **~$25/mo** and up at moderate MAU
Groq	API credits / rate limits	Heavy API usage	Pay-as-you-go; depends on assessment volume
Resend	Free email tier	Volume	From **~$20/mo** at higher volumes (verify)

This repo does not contain Razorpay/Stripe SDKs in `package.json` — payment is described in UI; **production payment integration** may still be pending.

---

3. Folder & File Structure

Below: **every file** from a clean tree (excluding `node_modules`, `.next`, `.git`). **Critical** files marked 🔴.

Path	Purpose
🔴 `package.json`	Dependencies & npm scripts — **critical**
🔴 `package-lock.json`	Locked dependency versions — **critical**
🔴 `tsconfig.json`	TypeScript config — **critical**
🔴 `next.config.ts`	Next.js config (e.g. remote images) — **critical**
🔴 `proxy.ts`	Clerk + nutritionist cookie middleware — **critical** (routing)
`postcss.config.mjs`	PostCSS for Tailwind — **critical** for build
`eslint.config.mjs`	ESLint rules
`next-env.d.ts`	Next.js TypeScript refs

`vercel.json`	Vercel cron schedule for `/api/reminders/cron`
`.gitignore`	Git ignore rules
`.cursorrules`	Cursor editor rules
`AGENTS.md`, `CLAUDE.md`	Agent / assistant instructions
`README.md`	Default Next.js readme (minimal project-specific docs)
`SUPABASE_SETUP.sql`	🔴 SQL bootstrap for tables — **critical** reference for schema
`components.json`	shadcn/ui config
🔴 `app/layout.tsx`	Root layout + ClerkProvider — **critical**
`app/globals.css`	Global CSS / Tailwind
`app/page.tsx`	Landing page composition
`app/favicon.ico`	Favicon
🔴 `app/sign-in/[[...sign-in]]/page.tsx`	Patient Clerk sign-in + nutritionist custom flow — **critical**
`app/assessment/page.tsx`	Assessment questionnaire UI
`app/assessment/results/page.tsx`	Assessment results display
`app/assessment/purchase/page.tsx`	₹29 purchase / lead flow (Clerk-gated)
`app/booking/page.tsx`	🔴 Booking hub: Clerk redirect & routing — **critical**
`app/booking/onboard/page.tsx`	Patient profile onboarding steps
`app/booking/dashboard/page.tsx`	Patient session dashboard
`app/booking/new/page.tsx`	Book new appointment
`app/booking/profile/page.tsx`	Patient profile & bookings management
`app/booking/success/page.tsx`	Post-booking confirmation
`app/booking/purchase/page.tsx`	Extra booking purchase page (if used)
`app/nutritionist-dashboard/page.tsx`	🔴 Nutritionist main dashboard — **critical**
`app/nutritionist-dashboard/availability/page.tsx`	Nutritionist availability editor
`app/nutritionist-update-password/page.tsx`	Supabase password recovery completion
`app/nutritionist/page.tsx`	Legacy nutritionist route (redirects / older UI)
`app/nutritionist/availability/page.tsx`	Legacy availability
`app/nutritionist-login/page.tsx`	Alternate nutritionist login entry
🔴 `app/api/assessment/route.ts`	AI assessment report API — **critical**

🔴 `app/api/save-lead/route.ts`	Lead capture + Resend — **critical**
🔴 `app/api/reminders/cron/route.ts`	Scheduled reminder processing — **critical** for ops
`components/PageLoader.tsx`	Page load animation
`components/ScrollProgress.tsx`	Scroll progress bar
`components/sections/*`	Landing sections (Hero, Navbar, Pricing, FAQ, …)
`components/ui/button.tsx`, `CalendarPicker.tsx`	Shared UI
🔴 `lib/booking-actions.ts`	Server actions: clients, appointments, leads — **critical**
🔴 `lib/nutritionist-actions.ts`	Nutritionist dashboard server actions — **critical**
🔴 `lib/nutritionist-config.ts`	**Whitelist emails** — **critical**
🔴 `lib/supabase.ts`	Browser Supabase client — **critical**
🔴 `lib/supabase-admin.ts`	Service role client (server only) — **critical**
`lib/utils.ts`	`cn()` helpers
`public/*`	Static SVG assets
`.env.local`	**Secrets (not committed)** — local env

Never delete: `package.json`, `proxy.ts`, `app/layout.tsx`, `lib/supabase*.ts`, `lib/booking-actions.ts`, `lib/nutritionist-actions.ts`, `lib/nutritionist-config.ts`, `SUPABASE_SETUP.sql` (reference), core `app/booking/*` and `app/sign-in/*` without replacement.

---

4. Database Schema (Supabase)

Source of truth in repo: `SUPABASE_SETUP.sql` + TypeScript types / inserts in `lib/booking-actions.ts`.
Note: There is **no** `profiles` or `bookings` table in this codebase — the app uses **`clients`** and **`appointments`**.

Tables

`clients` (patient / plan holder)

Aligned with `ClientRow` in `lib/booking-actions.ts` and SQL migration snippet:

Column (inferred)	Type	Purpose
`id`	UUID	Primary key
`clerk_user_id`	TEXT UNIQUE	Clerk user id
`name`	TEXT	Display name
`email`	TEXT	Email
`phone`	TEXT	Phone (SQL adds column)
`plan_start_date`	DATE	Plan start

`plan_end_date`	DATE	Plan end
`sessions_total`	INTEGER	Total sessions in plan
`sessions_used`	INTEGER	Used count
`sessions_remaining`	INTEGER	Remaining
`status`	TEXT	`active` | `expired` | `completed`
`assessment_goal`	TEXT (optional)	Goal from onboarding

Relationships: Referenced by `appointments.client_id` .

`nutritionists`

Column	Type	Purpose
`id`	UUID PK	Nutritionist id
`clerk_user_id`	TEXT UNIQUE nullable	Legacy Clerk link
`name`	TEXT	Name
`email`	TEXT UNIQUE	Email
`bio`	TEXT	Bio
`created_at`	TIMESTAMPTZ	Created

`availability`

Column	Type	Purpose
`id`	UUID PK	Row id
`nutritionist_id`	UUID FK → `nutritionists.id`	Owner
`day_of_week`	INTEGER 0–6	Day
`start_time`, `end_time`	TIME	Window
`is_active`	BOOLEAN	Active flag

`appointments`

Column	Type	Purpose
`id`	UUID PK	Appointment id
`client_id`	UUID FK → `clients.id`	Client
`nutritionist_id`	UUID FK → `nutritionists.id`	Nutritionist
`session_number`	INTEGER	Session index

`scheduled_date`	DATE	Date
`scheduled_time`	TIME	Time
`reason`	TEXT	Reason / notes
`status`	TEXT	`pending`, `confirmed`, `rejected`, `completed`, `cancelled`
`notes`	TEXT	Notes
`reminder_24h_sent`, `reminder_1h_sent`	BOOLEAN	Reminder flags
`created_at`	TIMESTAMPTZ	Created

`leads` (inferred from code)

`save-lead` uses `upsert` on `email` → expect columns at minimum: `name`, `email`, `phone`, `source` (+ possibly timestamps). **Create/migrate via Supabase dashboard** if not present.

Row Level Security (RLS)

In `SUPABASE_SETUP.sql`, RLS is **commented out** (optional). **Default in many new projects: RLS off until enabled.**

Table	RLS in repo	Notes
`clients`, `appointments`, `nutritionists`, `availability`	**Not enabled in committed SQL**	App uses **service role** server-side → bypasses RLS if enabled without policies

Bold warning: Production should enable RLS + policies or strictly lock API access — current pattern relies on **server-only service role** and **Clerk server auth**.

Account deletion

Not implemented in reviewed code: No automated "delete user" pipeline found. **Clerk user deletion** and **Supabase auth user deletion** would need manual/admin steps; **client rows** would remain unless CASCADE or cleanup job is added.

---

5. Authentication Flow

Supabase Auth (nutritionists)

- Nutritionists authenticate via `supabase.auth.signInWithPassword` on the **sign-in page** (nutritionist tab).
- Session stored in **browser** (typical Supabase JS: **localStorage** for session persistence).
- `nut-email` **cookie** is set client-side after login so `proxy.ts` can recognize nutritionist traffic (middleware cannot read localStorage).

Whitelist

- **File:** `lib/nutritionist-config.ts` — `ALLOWED_NUTRITIONIST_EMAILS` + `isNutritionistEmail()`.
- **Client-side:** Sign-in UI checks before password step.

- **Server / API:** Email-based server actions in `lib/nutritionist-actions.ts` should only accept operations for matching nutritionist email (review when changing).

Session expiry

- **Clerk:** Session managed by Clerk; protected routes use `auth.protect()` in middleware.
- **Supabase:** Refresh token behavior per Supabase defaults; expired session → nutritionist dashboard redirects to `/sign-in`.

JWT storage and lifetime

- **Clerk:** Session cookies / tokens managed by Clerk SDK (**not documented in-repo**; see Clerk docs).
- **Supabase:** Access/refresh tokens in **localStorage** (browser client). **Exact TTL** = Supabase project settings.

Middleware / routing

- **File:** `proxy.ts` (Next.js uses this as the middleware entry in this project setup).
- **Nutritionist path:** If **no Clerk** `userId` but `nut-email` **cookie** matches whitelist → allow only `/nutritionist-dashboard`, `/sign-in`, `/nutritionist-update-password`, `/api/*`, `/_next/*`; else redirect to dashboard.
- **Clerk** `userId` **present:** Nutritionist branch **skipped** — patient flows apply.
- **Protected patient routes:** `createRouteMatcher` list → `auth.protect()`.

---

6. Every User Flow (Exact Routes)

A. Landing → Assessment → ₹29 → Login → Purchase

Step	Route	Checks / behavior
1	`/`	Public
2	`/assessment`	Public; answers submitted to API
3	`/assessment/results`	Reads `localStorage`
4	CTA may set `postLoginDest` / go to `/assessment/purchase`	Clerk required on purchase page
5	`/sign-in`	Clerk sign-in
6	`/booking`	Hub: `postLoginDest === '29-plan'` → `/assessment/purchase`
7	`/assessment/purchase`	Calls `/api/save-lead`

B. Landing → ₹3,999 → Login → Onboard → Book → Success

Step	Route	Checks
1	`/`	Public
2	`/booking`	Sign-in CTA → Clerk
3	`/booking` (signed in)	`getClientByClerkId` → no row → `/booking/onboard`
4	`/booking/onboard`	`createClientProfile` → then code pushes **`/booking/dashboard`** (verify current file)

5	`/booking/new`	Eligibility checks
6	`/booking/success`	Confirmation

C. Nutritionist

Step	Route	Checks
1	`/sign-in` (nutritionist tab)	Whitelist email; Supabase password
2	`/nutritionist-dashboard`	`getSession`; load data by email
3	`/nutritionist-dashboard/availability`	Same pattern

D. Logged-in patient → Landing

Step	Route	Behavior
1	`/`	Navbar shows **My Profile** / **My Sessions**; sections may personalize (e.g. Comparison, AssessmentHero)

---

7. Security Checklist — Current Status

Item	Status
Supabase RLS on all tables	⚠️ **Partial** — SQL shows RLS commented; not verified live
API keys not exposed in frontend	⚠️ **Partial** — `NEXT_PUBLIC_SUPABASE_ANON_KEY` is public by design; **service role must never be client**
Env vars for secrets	✅ **Done** (pattern); **`.env.local` not in git**
Nutritionist whitelist server-side	⚠️ **Partial** — whitelist in shared config; **`nut-email` cookie is not cryptographically bound to session** (spoofing risk for route gate)
No sensitive data in localStorage	❌ **Not done** — assessment results + meta in `localStorage`
Middleware protecting private routes	✅ **Done** (Clerk + nutritionist cookie logic)
Input validation on all forms	⚠️ **Partial** — some API validation; not exhaustive
Rate limiting on auth	❌ **Not done** in repo
HTTPS enforced	✅ typical on Vercel production
CORS	✅ same-origin API routes default

---

8. Security Issues Before Going Live

Issue	Risk (simple)	Fix	Priority
`nut-email` **cookie trust**	Anyone could set cookie to whitelisted email and hit routes	Bind to signed HttpOnly cookie server-side or use Supabase SSR middleware	**High**
Service role usage	Bypasses RLS; leaked key = full DB	Enable RLS + narrow policies; rotate keys; never expose	**High**
Assessment data in localStorage	XSS or shared device leaks health answers	Move to server-stored profile post-auth	**Medium**
No rate limit on APIs	Abuse / cost on Groq, spam leads	Edge rate limit, CAPTCHA on public POST	**High**
`/api/save-lead` **spam**	Email noise, DB fill	Auth or honeypot + validation	**Medium**
Cron endpoint	If `CRON_SECRET` weak/missing, job could be triggered	Enforce secret header check in route (verify implementation)	**High**

---

9. Tools, Services & Costs (Table)

Service	Role in project	Free limit (typical)	After free (typical)
Supabase	DB + nutritionist auth	Free tier limits	~**$25/mo** Pro + usage
Vercel	Hosting + cron	Hobby limits	Pro ~**$20+/mo**
Clerk	Patient auth	Free/dev MAU caps	~**$25/mo**+ by MAU
Groq	Assessment AI	Credits / rate limits	Pay-as-you-go
Resend	Email	Free tier caps	~**$20/mo**+
Cal.com (if used)	Scheduling	Free tier	Paid plans

Payment: No Stripe/Razorpay in `package.json` — add when integrating.

---

10. What to Learn to Maintain This Project

Tech	Key concept	Free resource	Time to basics
Next.js App Router	Server vs client components, routing	https://nextjs.org/docs	1–2 weeks
React	State, hooks, effects	https://react.dev	1–2 weeks
TypeScript	Types for APIs & props	https://www.typescriptlang.org/docs	1 week
Clerk	Sessions, middleware, `auth()`	https://clerk.com/docs	2–4 days
Supabase	Auth + Postgres + service role vs anon	https://supabase.com/docs	1 week

Tailwind	Utility-first styling	https://tailwindcss.com/docs	3–5 days
Vercel	Deploy + env + cron	https://vercel.com/docs	1–2 days

---

11. Before Going Live — Checklist

- ☐ All **environment variables** set in production (Clerk, Supabase, Groq, Resend, CRON secret, etc.)
- ☐ **Supabase RLS** reviewed and tested
- ☐ **Custom domain** + SSL
- ☐ **Payment gateway** live keys (when integrated)
- ☐ **Error tracking** (Sentry or similar)
- ☐ **Test data** removed or isolated
- ☐ **Supabase auth email** templates branded
- ☐ **Privacy Policy** & **Terms** pages
- ☐ **Compliance** (GDPR / India DPDP) reviewed with counsel
- ☐ **Load test** critical paths

---

12. Future Features & Needs

Feature	Likely needs
WhatsApp notifications	WhatsApp Business API / Twilio; new tables for message log; opt-in consent
Payment refunds	Payment provider APIs; `payments` table; admin workflow
Multiple nutritionists	Role assignment on booking; UI to pick nutritionist; possibly `nutritionist_id` on client
Rescheduling	Update `appointments` rules; audit log; notification hooks
Progress tracking	New `metrics` / `check_ins` tables; charts; client dashboard APIs

---

Critical warnings (bold)

- **Do not commit `SUPABASE_SERVICE_ROLE_KEY` or `CLERK_SECRET_KEY` to git.**
- **The `nut-email` cookie must not be the long-term security boundary for nutritionists.**
- **RLS is not proven enabled from SQL file — verify in Supabase dashboard.**

End of document.